

Phase 1 Assignment 1B - Backend API Projects

Goal: Master Phase 1 by building realistic backend APIs from a blank FastAPI project. Scope: Only use concepts from Phase 1 lessons 1-8. Standard: Portfolio-style backend practice, but implemented with beginner-safe Phase 1 tools.

Allowed Phase 1 Concepts

Use only:

- `FastAPI()`
- `@app.get`, `@app.post`, `@app.put`, `@app.delete`
- Path parameters with type hints and `Path()`
- Query parameters with defaults, optional values, lists, and `Query()`
- Request bodies with Pydantic `BaseModel`
- Field validation with `Field()`
- Nested Pydantic models
- Combining path, query, and body parameters
- `Body()` and `embed=True` if needed
- Returning dictionaries, lists, and Pydantic models
- `status_code=...`
- `HTTPException`
- `JSONResponse` only when a normal `return dict` is not enough
- The injected `Response` object for dynamic status codes or simple headers
- JSON file storage using Python's standard `json` module
- Python dictionaries/lists while processing loaded JSON data
- Basic Python business logic

Do not use:

- Databases, SQLAlchemy, Alembic, SQLModel, MongoDB, Redis
- `APIRouter`
- Dependency injection with `Depends`
- Middleware
- Authentication, authorization, JWT, OAuth2, password hashing, sessions
- `response_model`
- Pydantic custom validators such as `@field_validator` or `@model_validator`
- Form data or file uploads
- Headers/cookies as request inputs
- Templates, static files, frontend pages
- Background tasks, WebSockets, caching, rate limiting
- Pytest or automated testing frameworks

- Docker, deployment, environment configuration

Important: Use local JSON files for storage. Data should remain available after the server restarts.

General Rules For All Projects

- Build backend APIs only.
- Do not write a frontend.
- Do not copy solution code from the internet.
- Every project must run with `uvicorn main:app --reload`.
- Every resource should have a clear ID.
- Return proper HTTP status codes.
- Use `HTTPException` for errors such as not found, duplicates, invalid transitions, or business rule violations.
- Use Pydantic models for request bodies.
- Use `Field()`, `Path()`, and `Query()` validation wherever appropriate.
- Store and retrieve application data from JSON files.
- Load JSON data at the start of each request or through a small helper function.
- Save JSON data after every create, update, delete, or workflow action.
- Use simple `page` and `limit` query parameters only if you implement pagination. Do not use cursor pagination.
- Keep all endpoint implementations in `main.py`. Splitting into routers is a Phase 2 topic.

Recommended JSON storage rules:

- Put JSON files inside a `data/` folder.
- Keep each resource type in its own JSON file.
- Store records as a list of objects.
- Generate IDs by reading the existing records and using the next available integer.
- If a JSON file does not exist yet, create it with an empty list or seeded records.
- Do not use a database, ORM, or external storage service.
- Do not use concurrent file locking or advanced persistence patterns. This assignment is for local learning.
- Use helper functions such as `load_json(filename)` and `save_json(filename, data)` if you want to avoid repeated code.

Recommended pagination rules when used:

- `page`: default 1, minimum 1
- `limit`: default 10, minimum 1, maximum 100
- Response should include `items`, `page`, `limit`, and `total`

Project 1 - Neighborhood Services Directory API

Difficulty

Medium

Problem Statement

Build an API for a neighborhood services directory where users can discover local service providers such as plumbers, tutors, electricians, cleaners, mechanics, and designers.

The API should allow service providers to be created, searched, filtered, updated, reviewed, and removed. The goal is to practice resource design, path/query/body parameters, validation, and simple business rules.

Functional Requirements

- Create service provider profiles.
- List providers with search, filtering, sorting, and pagination.
- Fetch one provider by ID.
- Fully update a provider profile.
- Delete a provider.
- Add reviews to providers.
- List reviews for one provider.
- Return provider summary information, including average rating and review count.

API Requirements

- Store providers in `data/providers.json`.
- Store reviews in `data/reviews.json`.
- Load providers and reviews from JSON files when handling requests.
- Save providers and reviews back to JSON files after create, update, delete, and review actions.
- Provider IDs must be generated by the API.
- Review IDs must be generated by the API.
- Search should be case-insensitive.
- Filtering should happen through query parameters.
- Do not use authentication. Assume all requests are public.

Required Endpoints

Method	Path	Purpose
POST	<code>/providers</code>	Create a provider
GET	<code>/providers</code>	List/search/filter providers

GET	/providers/{provider_id}	Get one provider
PUT	/providers/{provider_id}	Replace provider details
DELETE	/providers/{provider_id}	Delete provider
POST	/providers/{provider_id}/reviews	Add a review
GET	/providers/{provider_id}/reviews	List reviews for a provider
GET	/categories	List available provider categories

Request And Response Expectations

POST /providers request should include:

- name
- category
- city
- area
- phone
- hourly_rate
- years_experience
- available
- skills

Successful POST /providers response:

- Status code 201
- Created provider object
- Generated id
- average_rating initialized to 0
- review_count initialized to 0

GET /providers should support:

- q: optional search by provider name or skill
- category: optional category filter
- city: optional city filter
- available: optional boolean filter
- min_rate: optional minimum hourly rate
- max_rate: optional maximum hourly rate
- min_experience: optional minimum years of experience
- page: optional pagination page
- limit: optional pagination limit

Successful list response should include:

- items

- page
- limit
- total
- filters_applied

POST /providers/{provider_id}/reviews request should include:

- reviewer_name
- rating
- comment

Successful review response:

- Status code 201
- Created review object
- Updated provider rating should be reflected when fetching the provider again

Validation Requirements

- provider_id: integer, greater than or equal to 1
- name: 2-80 characters
- category: one of a fixed set such as plumber, electrician, cleaner, tutor, mechanic, designer
- city: 2-60 characters
- area: 2-80 characters
- phone: must match a simple phone pattern, for example digits plus optional +, spaces, or dashes
- hourly_rate: greater than 0
- years_experience: greater than or equal to 0
- skills: list of strings, at least one skill required
- reviewer_name: 2-60 characters
- rating: integer from 1 to 5
- comment: 5-500 characters
- page: greater than or equal to 1
- limit: between 1 and 100

Business Rules

- Provider category must be from the allowed category list.
- Provider names do not need to be globally unique.
- A provider cannot be deleted if they have more than 10 reviews. Return a conflict error.
- Average rating must be recalculated after every new review.
- min_rate cannot be greater than max_rate.
- If no providers match a filter/search, return an empty items list, not an error.
- Deleting a provider should also remove or ignore its reviews in future review listing responses.

Edge Cases

- Fetching a missing provider should return 404.
- Updating a missing provider should return 404.
- Adding a review to a missing provider should return 404.
- Creating a provider with an invalid category should return validation or business-rule error.
- GET `/providers?available=maybe` should fail validation.
- GET `/providers?min_rate=100&max_rate=50` should return a clear error.
- Deleting the same provider twice should return 404 the second time.

Expected Folder Structure

```
neighborhood_services_api/  
  main.py  
  requirements.txt  
  README.md  
  data/  
    providers.json  
    reviews.json
```

Deliverables

- A runnable FastAPI app.
- Swagger UI should show all endpoints clearly.
- A short README explaining how to run the app.
- At least 8 sample requests documented in the README.
- JSON seed data for at least 5 providers and 5 reviews.
- Data should still exist after stopping and restarting the server.

Bonus Challenges

- Add `sort_by` query parameter with allowed values: `hourly_rate`, `years_experience`, `rating`.
- Add `order` query parameter with allowed values: `asc`, `desc`.
- Add a GET `/providers/top-rated` endpoint that returns providers with average rating greater than or equal to a query parameter.
- Add a GET `/providers/{provider_id}/summary` endpoint that returns only summary fields.

Project 2 - Inventory And Purchase Order API

Difficulty

Hard

Problem Statement

Build an inventory API for a small warehouse that manages products, suppliers, and purchase orders.

This project simulates common backend interview problems: unique fields, stock calculations, nested request bodies, filtering, order state changes, and conflict handling.

Functional Requirements

- Create and manage products.
- Create and manage suppliers.
- List products with filtering/searching/pagination.
- Create purchase orders containing multiple line items.
- Receive purchase orders and update stock.
- Cancel purchase orders.
- Track low-stock products.
- Calculate order totals from product prices and quantities.

API Requirements

- Store products in `data/products.json`.
- Store suppliers in `data/suppliers.json`.
- Store purchase orders in `data/purchase_orders.json`.
- Load data from JSON files when handling requests.
- Save data back to JSON files after create, update, delete, receive, and cancel actions.
- Do not accept client-provided IDs.
- `sku` must be unique across products.
- Purchase orders must contain at least one item.
- Order item product IDs must reference existing products.
- Supplier IDs must reference existing suppliers.

Required Endpoints

Method	Path	Purpose
POST	<code>/products</code>	Create a product
GET	<code>/products</code>	List/search/filter products

GET	/products/{product_id}	Get one product
PUT	/products/{product_id}	Replace a product
DELETE	/products/{product_id}	Delete a product
POST	/suppliers	Create a supplier
GET	/suppliers	List suppliers
GET	/suppliers/{supplier_id}	Get one supplier
POST	/purchase-orders	Create a purchase order
GET	/purchase-orders	List purchase orders
GET	/purchase-orders/{order_id}	Get one purchase order
POST	/purchase-orders/{order_id}/receive	Receive an order and increase stock
POST	/purchase-orders/{order_id}/cancel	Cancel an order
GET	/inventory/low-stock	List low-stock products

Request And Response Expectations

POST /products request should include:

- name
- sku
- category
- unit_price
- stock_quantity
- reorder_level
- active

Successful product response:

- Status code 201
- Generated id
- All submitted product fields

GET /products should support:

- q: optional search by product name or SKU
- category: optional filter
- active: optional boolean filter
- low_stock: optional boolean filter
- min_price: optional minimum price
- max_price: optional maximum price

- page
- limit

POST /suppliers request should include:

- name
- contact_email
- phone
- city
- active

POST /purchase-orders request should include:

- supplier_id
- items: list of order items
- notes: optional text

Each order item should include:

- product_id
- quantity

Successful purchase order response:

- Status code 201
- Generated id
- status set to draft
- items with product IDs, quantities, and calculated line totals
- total_amount

POST /purchase-orders/{order_id}/receive response:

- Status code 200
- Updated order with status set to received
- Product stock quantities increased by order quantities

POST /purchase-orders/{order_id}/cancel response:

- Status code 200
- Updated order with status set to cancelled

Validation Requirements

- product_id, supplier_id, order_id: integer, greater than or equal to 1
- name: 2-100 characters
- sku: 3-30 characters, uppercase letters, numbers, and dashes only
- category: 2-50 characters
- unit_price: greater than 0
- stock_quantity: greater than or equal to 0
- reorder_level: greater than or equal to 0
- contact_email: simple email pattern

- `phone`: simple phone pattern
- `city`: 2-60 characters
- `quantity`: integer, greater than 0
- `notes`: optional, maximum 500 characters
- `page`: greater than or equal to 1
- `limit`: between 1 and 100

Business Rules

- Duplicate SKU should return 409.
- Product cannot be deleted if it appears in an existing purchase order.
- Inactive suppliers cannot be used for new purchase orders.
- Inactive products cannot be added to purchase orders.
- Purchase order total must be calculated by the server, not trusted from the client.
- A draft order can be received.
- A draft order can be cancelled.
- A received order cannot be cancelled.
- A cancelled order cannot be received.
- Receiving an order should update stock only once.
- `low_stock=true` should return products where `stock_quantity <= reorder_level`.
- `min_price` cannot be greater than `max_price`.

Edge Cases

- Creating a product with an existing SKU should return 409.
- Creating an order with missing product IDs should return 404.
- Creating an order with a missing supplier ID should return 404.
- Receiving a missing order should return 404.
- Receiving an already received order should return 409.
- Cancelling an already received order should return 409.
- Listing products when none exist should return an empty list.
- Deleting a missing product should return 404.
- Invalid query values should produce FastAPI validation errors.

Expected Folder Structure

```
inventory_purchase_order_api/  
  main.py  
  requirements.txt  
  README.md  
  data/
```

```
products.json
suppliers.json
purchase_orders.json
```

Deliverables

- A runnable FastAPI app.
- At least 6 seeded products.
- At least 3 seeded suppliers.
- At least 2 seeded purchase orders.
- Seed data should be stored in JSON files.
- Data should still exist after stopping and restarting the server.
- README with request examples for product creation, filtering, order creation, receiving, and cancelling.
- Clear error responses for missing resources and invalid state transitions.

Bonus Challenges

- Add `GET /inventory/valuation` that returns total inventory value.
- Add `GET /products/{product_id}/movement-summary` that shows starting stock, ordered quantity, received quantity, and current stock using only JSON-stored data.
- Add `GET /purchase-orders?status=draft` filtering.
- Add query sorting for products by `name`, `unit_price`, or `stock_quantity`.

Project 3 - Helpdesk Ticket Management API

Difficulty

Advanced

Problem Statement

Build a backend API for a helpdesk system where customers submit support tickets and support agents manage ticket status, priority, and comments.

This project focuses on real-world workflows, nested data, status transitions, filtering, searching, and strict validation without using authentication or a database.

Functional Requirements

- Create customer records.

- Create support tickets for customers.
- List tickets with filters, search, and pagination.
- Fetch ticket details.
- Replace ticket details.
- Add internal or public comments to tickets.
- Change ticket status through controlled workflow endpoints.
- Assign tickets to support agents by name.
- Delete tickets only when allowed by business rules.
- Provide a ticket dashboard summary.

API Requirements

- Store customers in `data/customers.json`.
- Store tickets in `data/tickets.json`.
- Load customers and tickets from JSON files when handling requests.
- Save customers and tickets back to JSON files after create, update, delete, comment, assignment, and workflow actions.
- Use nested Pydantic models for customer contact details and ticket comments.
- No authentication. Treat `agent_name` as a normal request field where needed.
- Ticket IDs and customer IDs must be generated by the API.
- Ticket status must be controlled by business rules.

Required Endpoints

Method	Path	Purpose
POST	<code>/customers</code>	Create a customer
GET	<code>/customers</code>	List/search customers
GET	<code>/customers/{customer_id}</code>	Get one customer
PUT	<code>/customers/{customer_id}</code>	Replace customer details
POST	<code>/tickets</code>	Create a ticket
GET	<code>/tickets</code>	List/search/filter tickets
GET	<code>/tickets/{ticket_id}</code>	Get one ticket
PUT	<code>/tickets/{ticket_id}</code>	Replace editable ticket fields
DELETE	<code>/tickets/{ticket_id}</code>	Delete a ticket when allowed
POST	<code>/tickets/{ticket_id}/comments</code>	Add a ticket comment

POST	/tickets/{ticket_id}/assign	Assign ticket to an agent
POST	/tickets/{ticket_id}/start	Move ticket from open to in progress
POST	/tickets/{ticket_id}/resolve	Resolve ticket
POST	/tickets/{ticket_id}/reopen	Reopen resolved ticket
GET	/dashboard/tickets	Return ticket counts by status and priority

Request And Response Expectations

POST /customers request should include:

- name
- email
- phone
- company
- contact_address as a nested object

contact_address should include:

- city
- country
- postal_code

Successful customer response:

- Status code 201
- Generated id
- Customer details

POST /tickets request should include:

- customer_id
- subject
- description
- category
- priority

Successful ticket response:

- Status code 201
- Generated id
- status set to open
- comments initialized as empty list
- assigned_agent initialized as null or equivalent empty value

GET /tickets should support:

- q: optional search by subject or description
- status: optional status filter

- **priority:** optional priority filter
- **category:** optional category filter
- **customer_id:** optional customer filter
- **assigned_agent:** optional agent filter
- **page**
- **limit**

POST /tickets/{ticket_id}/comments request should include:

- **author_name**
- **visibility:** public or internal
- **message**

POST /tickets/{ticket_id}/assign request should include:

- **agent_name**

POST /tickets/{ticket_id}/resolve request should include:

- **resolution_note**
- **agent_name**

Validation Requirements

- **customer_id, ticket_id:** integer, greater than or equal to 1
- **Customer name:** 2-100 characters
- **Customer email:** simple email pattern
- **Customer phone:** simple phone pattern
- **company:** optional, maximum 100 characters
- **city:** 2-80 characters
- **country:** 2-80 characters
- **postal_code:** 3-20 characters
- **subject:** 5-150 characters
- **description:** 20-2000 characters
- **category:** one of billing, technical, account, general
- **priority:** one of low, medium, high, urgent
- **status:** one of open, in_progress, resolved
- **author_name:** 2-80 characters
- **message:** 2-1000 characters
- **agent_name:** 2-80 characters
- **resolution_note:** 10-1000 characters
- **page:** greater than or equal to 1
- **limit:** between 1 and 100

Business Rules

- A ticket must reference an existing customer.
- New tickets always start as `open`.
- Editable ticket fields are `subject`, `description`, `category`, and `priority`.
- `PUT /tickets/{ticket_id}` must not allow direct status changes. Status changes happen through workflow endpoints.
- Open tickets can move to `in_progress`.
- In-progress tickets can move to `resolved`.
- Open tickets can also be resolved directly if a resolution note is provided.
- Resolved tickets can be reopened.
- Resolved tickets cannot be assigned to a new agent unless reopened first.
- Tickets with comments cannot be deleted.
- Urgent tickets must have priority displayed clearly in list and detail responses.
- Dashboard summary should count tickets by status and priority.

Edge Cases

- Creating a ticket for a missing customer should return 404.
- Starting a missing ticket should return 404.
- Starting an already resolved ticket should return 409.
- Resolving an already resolved ticket should return 409.
- Reopening an open ticket should return 409.
- Assigning a resolved ticket should return 409.
- Deleting a ticket with comments should return 409.
- Listing tickets with filters that match nothing should return an empty list.
- Invalid status or priority query values should fail validation.

Expected Folder Structure

```
helpdesk_ticket_api/  
  main.py  
  requirements.txt  
  README.md  
  data/  
    customers.json  
    tickets.json
```

Deliverables

- A runnable FastAPI app.
- At least 5 seeded customers.
- At least 8 seeded tickets across different statuses and priorities.

- Seed data should be stored in JSON files.
- Data should still exist after stopping and restarting the server.
- README with sample workflows:
- Create customer
- Create ticket
- Assign ticket
- Add comment
- Resolve ticket
- Reopen ticket
- Clear dashboard response from `/dashboard/tickets`.

Bonus Challenges

- Add `GET /customers/{customer_id}/tickets`.
- Add `GET /tickets/urgent` for urgent unresolved tickets.
- Add `GET /tickets/{ticket_id}/timeline` showing status changes and comments in order.
- Add `include_internal` boolean query parameter to ticket detail so internal comments can be hidden or shown.

Project 4 - Restaurant Ordering And Kitchen Workflow API

Difficulty

Very Advanced

Problem Statement

Build a backend API for a restaurant that manages menu items, dining tables, customer orders, kitchen workflow, and receipts.

This is the most complex Phase 1 project. It requires careful REST design, nested request bodies, query filters, calculated fields, state transitions, and many business rules while still using only Phase 1 concepts.

Functional Requirements

- Create and manage menu items.
- List menu items with filters/search/pagination.
- Create dining tables.
- Open customer orders for tables.

- Add items to an order.
- Replace an order before it is submitted.
- Submit an order to the kitchen.
- Move order through kitchen statuses.
- Cancel orders when allowed.
- Generate an order receipt.
- List orders by table, status, or total range.

API Requirements

- Store menu items in `data/menu_items.json`.
- Store tables in `data/tables.json`.
- Store orders in `data/orders.json`.
- Load menu items, tables, and orders from JSON files when handling requests.
- Save data back to JSON files after create, update, delete, submit, prepare, serve, and cancel actions.
- Menu item IDs, table IDs, and order IDs must be generated by the API.
- Prices and totals must be calculated by the server.
- Order item request bodies should use nested Pydantic models.
- No authentication, no payments integration, no database.
- No partial update endpoints. Use `PUT` for full replacement where required.

Required Endpoints

Method	Path	Purpose
POST	<code>/menu-items</code>	Create a menu item
GET	<code>/menu-items</code>	List/search/filter menu items
GET	<code>/menu-items/{item_id}</code>	Get one menu item
PUT	<code>/menu-items/{item_id}</code>	Replace a menu item
DELETE	<code>/menu-items/{item_id}</code>	Delete a menu item
POST	<code>/tables</code>	Create a dining table
GET	<code>/tables</code>	List tables
GET	<code>/tables/{table_id}</code>	Get one table
POST	<code>/orders</code>	Open a new order
GET	<code>/orders</code>	List/search/filter orders
GET	<code>/orders/{order_id}</code>	Get one order

PUT	/orders/{order_id}	Replace draft order items
POST	/orders/{order_id}/submit	Submit draft order to kitchen
POST	/orders/{order_id}/prepare	Mark submitted order as preparing
POST	/orders/{order_id}/serve	Mark prepared order as served
POST	/orders/{order_id}/cancel	Cancel order when allowed
GET	/orders/{order_id}/receipt	Get calculated receipt

Request And Response Expectations

POST /menu-items request should include:

- name
- category
- description
- price
- available
- tags

Successful menu item response:

- Status code 201
- Generated id
- All submitted fields

GET /menu-items should support:

- q: optional search by name, description, or tag
- category: optional filter
- available: optional boolean filter
- min_price
- max_price
- page
- limit

POST /tables request should include:

- table_number
- capacity
- section

Successful table response:

- Status code 201
- Generated id
- occupied initialized to false

POST /orders request should include:

- `table_id`
- `customer_name`: optional
- `items`: list of order item objects

Each order item object should include:

- `menu_item_id`
- `quantity`
- `special_instructions`: optional

Successful order response:

- Status code 201
- Generated `id`
- status set to draft
- Server-calculated item prices
- Server-calculated subtotal
- Server-calculated service charge
- Server-calculated total

PUT `/orders/{order_id}` request should replace the full draft order:

- `table_id`
- `customer_name`
- `items`

GET `/orders` should support:

- `status`: optional filter
- `table_id`: optional filter
- `min_total`: optional filter
- `max_total`: optional filter
- `page`
- `limit`

GET `/orders/{order_id}/receipt` should return:

- Restaurant receipt ID or order ID
- Table number
- Order status
- Line items with names, quantities, unit prices, and line totals
- Subtotal
- Service charge
- Total

Validation Requirements

- `item_id`, `table_id`, `order_id`: integer, greater than or equal to 1
- Menu item `name`: 2-100 characters

- category: one of starter, main, dessert, drink, side
- description: 5-500 characters
- price: greater than 0
- tags: list of strings
- table_number: integer, greater than or equal to 1
- capacity: integer from 1 to 20
- section: one of indoor, outdoor, vip, bar
- customer_name: optional, 2-80 characters when provided
- menu_item_id: integer, greater than or equal to 1
- quantity: integer from 1 to 20
- special_instructions: optional, maximum 200 characters
- status: one of draft, submitted, preparing, served, cancelled
- page: greater than or equal to 1
- limit: between 1 and 100
- min_price cannot be greater than max_price
- min_total cannot be greater than max_total

Business Rules

- Orders must reference an existing table.
- Order items must reference existing menu items.
- Unavailable menu items cannot be added to new orders.
- Orders must contain at least one item.
- Draft orders can be replaced with PUT.
- Submitted, preparing, served, and cancelled orders cannot be replaced.
- Draft orders can be submitted.
- Submitted orders can move to preparing.
- Preparing orders can move to served.
- Draft and submitted orders can be cancelled.
- Preparing or served orders cannot be cancelled.
- A table becomes occupied when an order is submitted.
- A table becomes available again when an order is served or cancelled.
- A table cannot have two active submitted/preparing orders at the same time.
- Menu items used in non-cancelled orders cannot be deleted.
- Subtotal is the sum of item quantity multiplied by menu item price.
- Service charge is 10 percent of subtotal.
- Total is subtotal plus service charge.
- Totals must be calculated by the API, not accepted from the client.

Edge Cases

- Creating an order for a missing table should return 404.
- Creating an order with a missing menu item should return 404.
- Creating an order with an unavailable menu item should return 409.
- Submitting a missing order should return 404.
- Submitting an already submitted order should return 409.
- Preparing a draft order should return 409.
- Serving a submitted order before preparing should return 409.
- Cancelling a preparing or served order should return 409.
- Replacing a non-draft order should return 409.
- Deleting a menu item used by an active order should return 409.
- Listing orders with filters that match nothing should return an empty list.
- Invalid boolean, integer, or enum-like query values should fail validation.

Expected Folder Structure

```
restaurant_ordering_api/
  main.py
  requirements.txt
  README.md
  data/
    menu_items.json
    tables.json
    orders.json
```

Deliverables

- A runnable FastAPI app.
- At least 10 seeded menu items.
- At least 5 seeded tables.
- At least 5 seeded orders across different statuses.
- Seed data should be stored in JSON files.
- Data should still exist after stopping and restarting the server.
- README with full workflow examples:
 - Create menu item
 - Create table
 - Create draft order
 - Replace draft order
 - Submit order
 - Prepare order
 - Serve order
 - View receipt

- Clear error handling for invalid state transitions.

Bonus Challenges

- Add `GET /tables/{table_id}/active-order`.
- Add `GET /orders/kitchen` to list only submitted and preparing orders.
- Add `GET /menu-items/popular` by counting menu item appearances in served orders.
- Add `discount_percent` as an optional body field on order creation, validated between 0 and 50, and calculate totals after discount.
- Add a dynamic `Response` header such as `X-Order-Status` when fetching an order.

Final Submission Checklist

For each project, submit:

- The project folder.
- `main.py`.
- `requirements.txt`.
- `README.md`.
- A list of all endpoints.
- All required JSON data files.
- At least 10 manual test cases with request path, request body if any, expected status code, and expected response shape.
- A short reflection explaining which Phase 1 concepts were used.

Before submitting, verify:

- The app starts without errors.
- `/docs` loads.
- All endpoints appear in Swagger UI.
- Invalid path parameters return validation errors.
- Invalid query parameters return validation errors.
- Invalid request bodies return validation errors.
- Missing resources return 404.
- Duplicate/conflicting actions return 409.
- Deletes that return 204 do not return a response body.
- Created, updated, and deleted data persists in JSON files after server restart.
- No Phase 2 or later features were used.

Difficulty Progression

Project	Difficulty	Main Skill Focus
Neighborhood Services Directory API	Medium	CRUD, filtering, search, reviews
Inventory And Purchase Order API	Hard	Nested bodies, unique fields, stock logic
Helpdesk Ticket Management API	Advanced	Workflow rules, comments, dashboard summaries
Restaurant Ordering And Kitchen Workflow API	Very Advanced	State transitions, calculated receipts, multi-resource rules

Complete the projects in order. Do not skip directly to the advanced projects unless the earlier ones feel easy from a blank file.

--	--	--